

Real-Time Graph Neural Network for Malicious Traffic Detection

Ahmed Salah Tawfik Ibrahim

PNTLab

CNIT

Pisa, Italy

ahmed.ibrahim@cnit.it

Emilio Paolini

TeCIP Institute

Scuola Superiore Sant'Anna

Pisa, Italy

emilio.paolini@santannapisa.it

Filippo Cugini

PNTLab

CNIT

Pisa, Italy

filippo.cugini@cnit.it

Francesco Paolucci

PNTLab

CNIT

Pisa, Italy

francesco.paolucci@cnit.it

Abstract—Graph neural networks (GNNs) have been recently proposed as powerful tools for detecting cyberattacks, addressing the challenges in cybersecurity as traditional methods fail to keep up with the complexity and scale of modern threats. However, a solution that effectively balances performance, scalability, and practical usability in real-time scenarios is still lacking. Hence, in this work we propose a real-time GNN framework designed to detect malicious traffic with high speed and accuracy. To achieve this, network traffic is mapped onto a graph, with its properties optimized through experimental analysis. The framework is designed to balance accuracy and computational efficiency, addressing the scalability issues commonly encountered in real-time applications. This balance is reflected in the model's ability to achieve an F1 score of 0.9 or higher with a classification window starting from 100 packets. Furthermore, the proposed model is compared against a state-of-the-art approach, highlighting its strengths in terms of inference speed and detection performance. Additionally, a scalability-accuracy analysis shows that accurate detection can be achieved with as few as 50-75 packets for certain attack types, proving the feasibility of the approach in real-time scenarios.

Index Terms—Graph Neural Network, GNN, Cybersecurity

I. INTRODUCTION

Among the different applications of Artificial Intelligence (AI), cyberattack detection is an active area of research. Traditional approaches, such as rule-based methods and statistical models, while effective in the past, have gradually been discontinued due to their limitations in adapting to sophisticated and evolving attack patterns [1]. Indeed, these methods often rely on predefined signatures or static heuristics, limiting their effectiveness in complex network environments.

In recent years, Neural Networks (NNs) have been proposed as a potential alternative, leveraging their ability to learn complex patterns and adapt to dynamic environments. Among these, Graph Neural Networks (GNNs) show promising results when it comes to malicious traffic detection [2]. Unlike conventional NNs, GNNs are designed to operate on graph-structured data, enabling them to capture the intricate relationships and interactions inherent to network traffic. By leveraging graph structures, GNNs can model the connections between different hosts and flows, offering a deeper contextual understanding that traditional models cannot achieve [3].

Being an active area of research, several GNN models have been proposed for detecting various types of cyberat-

tacks. However, most of these recent works have primarily focused on the development and training of the models rather than addressing their scalability, real-time applicability, or deployment in practical scenarios. A key requirement for such models when deployed in real-time network traffic analysis is the detection inference responsiveness, ideally achieved by analyzing only a few packets per flow. This enables the timely mitigation of attacks, preventing them from impacting network resources or causing denial-of-service disruptions. In addition, due to the exploding usage of edge devices, typically characterized by limited computational power and energy resources, detection models must be agile and possibly lightweight to account for these limitations [4].

To meet the aforementioned requirements, this work introduces a novel method for constructing traffic graphs, addressing the challenges of deploying GNNs for real-time malicious traffic detection. Specifically, our graph construction approach transforms raw network traffic into a graph structure that effectively captures the temporal dynamics and flow relationships between packets. We then experimentally determine the minimum traffic observation window required for accurate classification. This ensures practicality in real-time applications by addressing scalability concerns often associated with large traffic graphs. Additionally, we benchmark our model against a state-of-the-art solution from the literature, focusing on creating a GNN that is compact, accurate, and optimized for fast inference.

II. LITERATURE REVIEW

A. Graph Neural Networks (GNNs)

Graph neural networks are neural networks that handle graph data in order to perform tasks such as node classification, edge prediction, or graph classification [5]. The basic idea is that if the data of interest can be modelled as a graph, then the GNN utilizes the graph structure to exchange messages between neighbouring nodes with the aim of aggregating their features to learn representations that are useful for the task at hand [6]. There are many types of GNNs depending on the way messages are exchanged. The most common ones are graph convolutional networks (GCNs) and graph attention networks (GATs). In a GCN layer, a node representation is computed by applying a learnable kernel to the aggregation of the node's

features with those of its neighbors and potentially the features of the edges connecting it to its neighbors. This means that the number of layers in a GCN corresponds to the radius of the neighborhood of nodes considered for aggregation. GATs are different from GCNs because they incorporate learnable attention weights in the aggregation process.

B. Cyberattack Detection

Historically, the problem of cyberattack detection has been addressed using different machine and deep learning models. For instance, [7] develop a number of machine learning models including a random forest, a support vector machine and a fully connected neural network. They perform a feature selection process before training their models on the CTU-13 dataset [8], which is considered outdated by today's standards.

Another example is the work by [9] where they build and evaluate a number of deep learning methods, namely LSTMs and its variants, against different attack scenarios. In [10], the authors propose real-time traffic classification using computer vision models, such as ResNet.

In general, these works had remarkable results in terms of the F1 score. However, they either suffer from outdated training data or lack the necessary analysis to determine how fast they are. Additionally, they do not capture information related to the network structure, which could be useful in detecting distributed attacks. This is where graph neural networks come to fill in the gap.

C. Cyberattack Detection using GNNs

The problem of detecting malicious traffic maps perfectly to GNNs. This is because network traffic can be represented as a graph in multiple ways. For example, the work in [11] represents packets exchanged between two endpoints as nodes and add edges between them based on their sequence of arrival and direction. They use the packet length as a scalar node feature and transform the malicious traffic detection problem into a graph classification problem. The works in [12] and [13] build on this and consider a two-level graph hierarchy. The first level, called the flow graph, represents packets exchanged between two endpoints and gets constructed following the method in [11]. The second level is the traffic graph which represents the network where each node represents the interaction between pairs of endpoints, and edges are added between nodes if they have an endpoint in common. The features of the nodes in the traffic graph are the learned representations of the corresponding flow graphs. The problem is then reduced to classifying the traffic graph nodes, where each interaction is classified as either malicious or benign. Both models achieve higher F1 scores than traditional methods, including classical neural networks.

III. METHODOLOGY

A. Graph Construction

The graph is constructed from raw traffic packets by representing each host as a node. Directed edges are added between nodes for every packet exchanged, denoted as $e_{i,j}$, where i and

j are the source and destination nodes respectively. This setup frames the problem as an edge classification task.

Since node features are not utilized and edge classification models are not standard in practice, the graph is transformed such that each edge in the original graph becomes a node in a new graph G' . Formally, let $G = (V, E)$ be the original graph with nodes V and edges E . In the transformed graph $G' = (V', E')$, the nodes V' correspond to the edges E in G , while the edges E' in G' encode relationships between these new nodes. Edges in G' are added as follows:

- **Temporal Sequence:** To capture the chronological order of packets within the same flow, edges are added between consecutive packets based on their arrival times. An undirected edge is created between two nodes (i.e., representing packets) v'_i and v'_{i+1} if they belong to the same flow and $t(v'_i) < t(v'_{i+1})$. Formally:

$$e' = (v'_i, v'_{i+1}) \quad \text{if } \text{Flow}(v'_i) = \text{Flow}(v'_{i+1}) \quad \text{and} \\ t(v'_i) < t(v'_{i+1}). \quad (1)$$

This ensures that the temporal dynamics of packet exchanged within each flow are preserved in the graph structure.

- **Flow Relationships:** Edges are added between nodes representing packets from different flows if their endpoints share a connection. Specifically:

$$e' = (v'_i, v'_j) \quad \text{if } \text{Dst}(v'_i) = \text{Src}(v'_j) \quad \text{or} \\ \text{Src}(v'_i) = \text{Dst}(v'_j). \quad (2)$$

This captures interactions between related flows, ensuring the graph encodes dependencies between shared endpoints.

Fig. 1 illustrates the graph construction process.

To ensure computational efficiency, two constraints are introduced:

- **Degree Limitation:** The degree of each node is limited to a maximum value, $d_{\max}$, determined experimentally.
- **Flow Compression:** An experimentally-determined number of packets from the same flow is aggregated into a single node, reducing the graph size while preserving critical information.

These optimizations ensure scalability and make the approach practical for real-time scenarios.

IV. TRAINING SETUP

This section discusses the dataset, the model and the training configuration used to evaluate the proposed method.

A. The Dataset

The selected dataset is the 5G-NIDD [14], a network intrusion detection dataset created from a real-world 5G test network, collected using the 5G Test Network (5GTN) in Oulu, Finland. This dataset combines both attack and benign traffic across various attack scenarios. Traffic generation involved real mobile devices connected to the 5GTN, ensuring authenticity

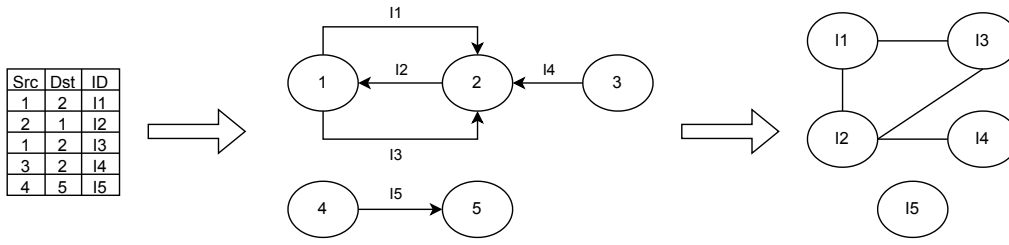


Fig. 1: Constructing the traffic graph

and relevance. The raw packets are used to build a dataset containing 11 packet features (i.e., *Length*, *TTL*, *Protocol*, *Layer*, *Flags*, *Transport Length*, *TCP Ack*, *TCP Flags*, *TCP Window Size*, *ICMP Type*, and *Transport Protocol*) in addition to the arrival time, source and destination addresses and the classification label. The dataset is then split into a training set and an evaluation set containing 70% and 30% of the original one respectively.

In order to ensure that packets from the entire lifespan of the traffic are represented in both sets, the dataset is split into timesteps of 1 second each. Afterwards, the split percentage of the training and evaluation sets is applied to each of these partial datasets. It is worth noting that the distribution of malicious and benign packets is preserved when performing the split. The evaluation set contains about 185000 packets belonging to benign flows and 116000 packets belonging to malicious ones. Graphs are then constructed following the methodology described in section III-A.

B. The Model

The exploited GNN to process this graph is a 6-layer GCN followed by a fully connected layer with dropout at the end for classification. For the first three convolutional layers, 32 units are exploited; while for the last three layers 8 units are used. The number of layers has been determined through a process of trial and error where it was determined that the minimum number of layers achieving high accuracy is 6. Similarly, different configurations of the convolutional layers' sizes have been tested to determine the best configuration. The goal of this model is to perform node classification where nodes are either singular packets or compressed packets belonging to the same flow.

In order to train the GNN, an Adam optimizer is used with a learning rate of 0.005 and a weight decay of 0.0005. The model is trained for a total of 30 epochs where it gets evaluated at the end of each epoch using the evaluation set. The version of the model that attains the highest F1 score throughout the training process is saved as the best-performing model.

V. EXPERIMENTS AND RESULTS

This section presents a comprehensive evaluation of the proposed model, focusing on the key parameters influencing its performance. The experiments are designed to determine the optimal values for critical hyperparameters, analyze trade-offs between F1 score and prediction time, and compare the

model's effectiveness against a state-of-the-art alternative. All experiments were conducted on an Intel(R) Xeon(R) Gold 6238R CPU of 2.20GHz.

A. Maximum Node Degree

The node degree is defined as the number of edges a node has in a graph. As mentioned in Sec.III-A, the maximum node degree in the input graphs to our model is to be determined experimentally. Therefore, the first experiment aims at measuring the F1 score and the prediction time for different graph sizes, with the graph size determined by (i) the number of nodes and (ii) the maximum node degree. We, therefore, train our model considering different configurations for the input graph where the number of nodes ranges between 10 and 100000 packets and the maximum degree ranges between 3 and 500.

Following the setup described in Sec.IV-B, each configuration defined by the number of nodes and their maximum degree is trained in order to record its best F1 score and the corresponding prediction time (i.e., the sum of the graph construction time and inference time). Then, these results are plotted in order to visually decide the optimal value for the maximum node degree as shown in Fig.2 and Fig.3. It can be observed that the F1 score starts to converge around the value of 50 edges per node. As for the prediction time, it does not

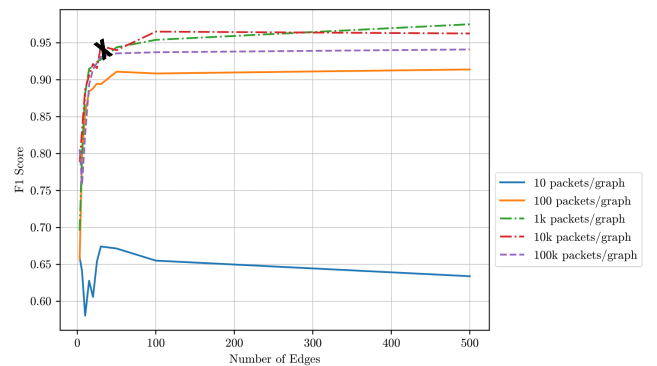


Fig. 2: F1 Score vs Maximum Number of Edges

undergo significant changes by varying the number of edges; however, it depends mainly on the number of nodes in the graph.

The tradeoff plot between the F1 score and the prediction time is considered. From Fig.4, it can be noted that the desired

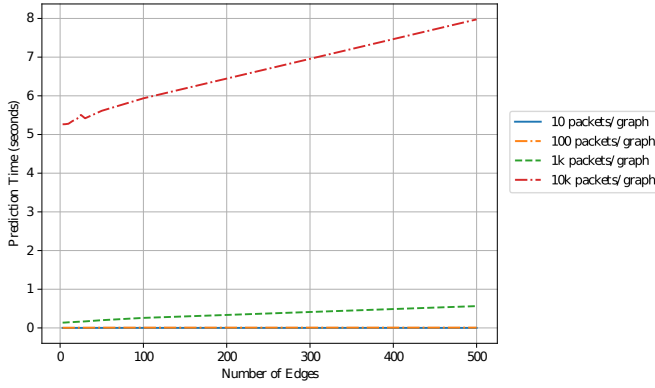


Fig. 3: Prediction Time vs Maximum Number of Edges

F1 score and prediction time can be achieved using a maximum number of edges of 10 or more. However, upon closer examination of the plot, it is evident that the desired trade-off between the F1 score and the prediction time happens when the maximum number of edges is 50 which is represented by the dash-dotted gray line. As the plot shows, going beyond

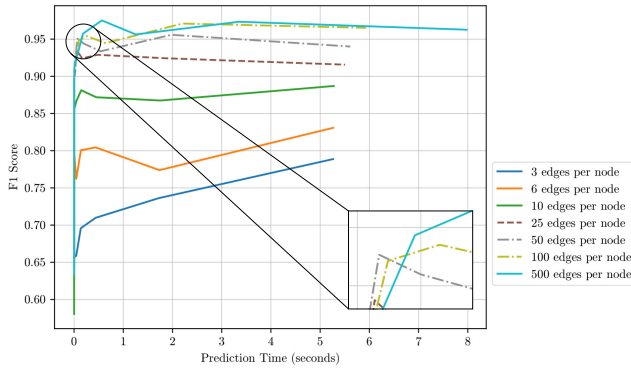


Fig. 4: Prediction Time vs F1 Score

50 edges per node does not lead to significant changes in the F1 score. On the other hand, it affects the prediction time negatively as it results in higher prediction times for graphs of the same number of nodes.

B. Optimal Node Granularity

The other parameter that is to be determined experimentally is the node granularity; i.e., how many packets belonging to the same flow can be compressed into a single node. Using the optimal maximum node degree from Sec.V-A, the graphs are constructed considering different granularity values. Again, following the outlined steps in Sec.IV-B, the best F1 score is recorded for each graph considering different classification windows; i.e., total number of packets, and different node granularities. The goal is to find a trade-off between the F1 score and the prediction time in order to choose the best node granularity.

Upon collecting the results, the trade-off plot between the F1 score and the prediction time is visualized in Fig.5a. It

shows that an acceptable F1 score can be achieved quickly with a granularity of 8. Upon deeper inspection of the plot (Fig.5b), it can be observed that the F1 score does not improve significantly if the granularity is 200. Even though the prediction time is not significantly affected by increasing the granularity, it is worth noting that it depends on the chosen aggregation method. For simplicity, we chose the mean when aggregating the features which is relatively fast to compute. If more complex aggregation methods are to be tested, then it is important to keep the number of aggregated nodes as low as possible. Therefore, the chosen optimal granularity is 8 instead of 200.

C. Attack Analysis

This experiment aims to test the number of packets our model needs to accurately detect different attacks. Different models were trained considering different classification windows for each. On average, they had a similar performance; therefore, we report here the results of the model trained with a classification window of 2000 packets. The test set used to evaluate the models during training was divided into a number of datasets such that each one would contain all the benign traffic in the original dataset in addition to the traffic related to only one attack type. Then, these datasets were transformed into graphs with different classification windows ranging between 3 and the classification window the model was trained on. So, for the 2000-packets model, the classification window size ranges between 3 and 2000.

The performance of the model with respect to each attack can be visualized in Fig. 6a. The model performs best at detecting Golden Eye attacks and Syn Flood attacks while struggling with Slow Loris attacks. Upon closer inspection in Fig.6b, it can be shown that the maximum F1 score for each attack can be achieved using a classification window size in the interval of 50 to 75 packets.

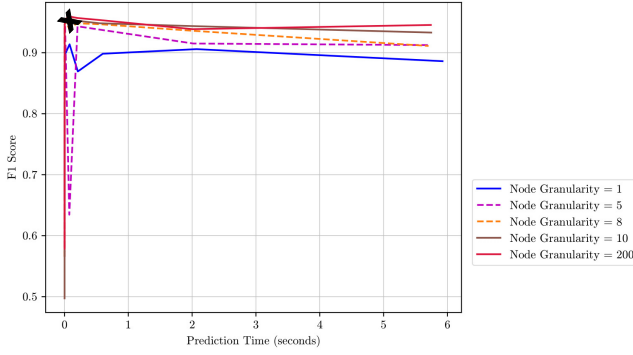
D. Comparison with FTG-Net

After choosing the best configuration for the graph, summarized in Table I, a GNN was trained and evaluated with the purpose of comparing it to the model proposed by [12].

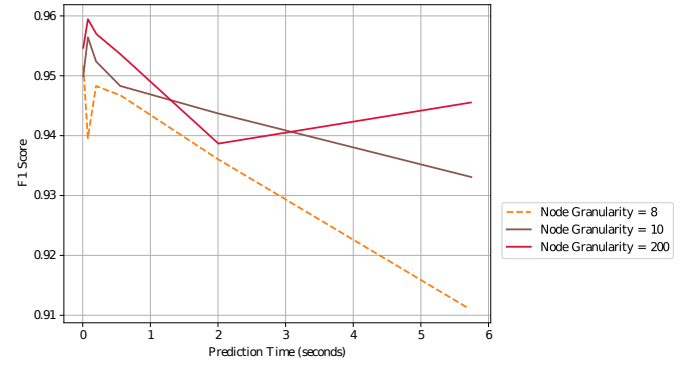
TABLE I: Summary of Experimental Configuration

Parameter	Optimal Value
Maximum Node Degree	50
Node Granularity (Packets per Node)	8
GNN Layers	6
Classification Window (Packets)	50-75

Our model generally achieves a higher F1 score (Fig. 7) while having a lower or similar prediction time for the small time windows (Fig. 8). This means that our model outperforms FTG-Net in a real-time attack detection scenario. It is also worth noting that, while FTG-Net has a relatively higher weighted F1 score of about 0.97, it has a generally unstable lower binary F1 score. In any case, our model still has a similar weighted F1 score of 0.93 which is more coherent with the binary F1 score.

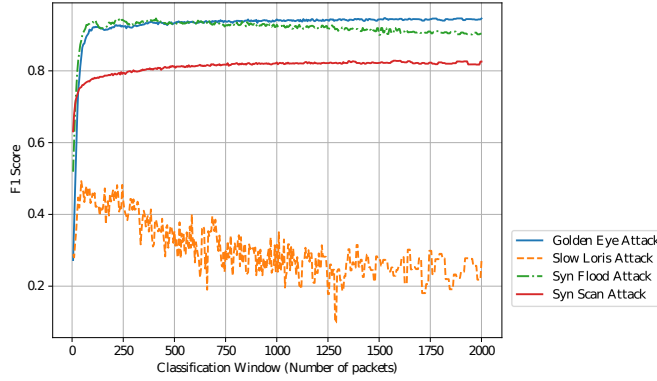


(a) F1 Score vs Prediction Time

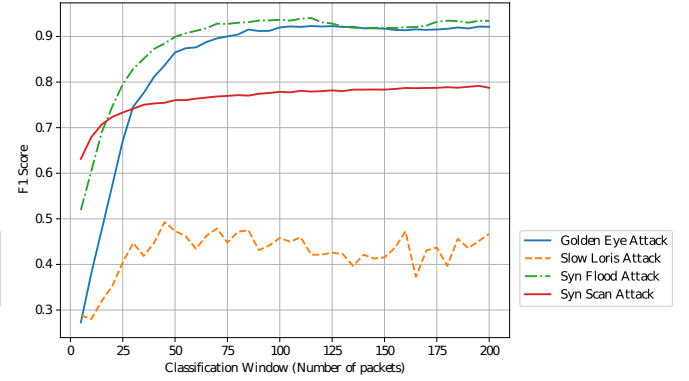


(b) Zooming in

Fig. 5: F1-Prediction Time Analysis with respect to Granularity



(a) F1 Score vs Classification Window



(b) Zooming in

Fig. 6: Attack Analysis

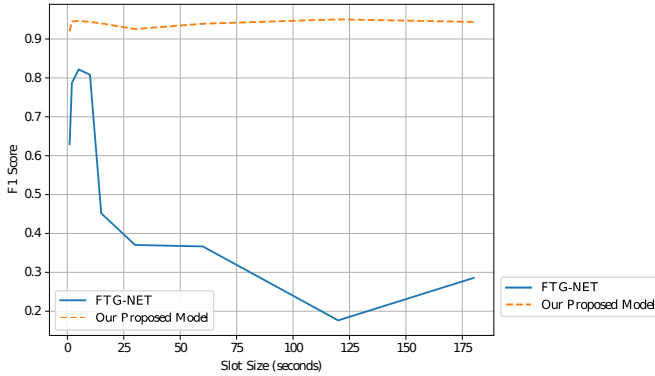


Fig. 7: FTG-Net vs Our Model (F1 Score)

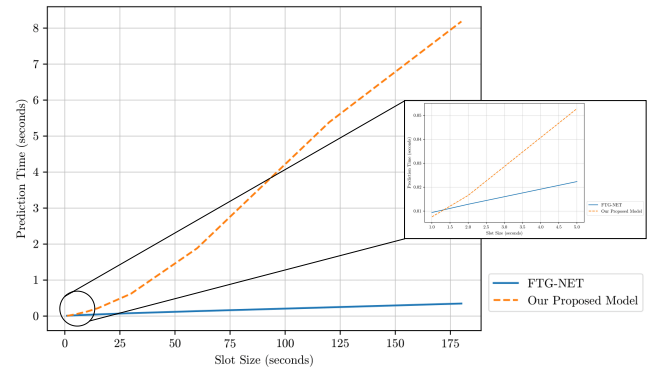


Fig. 8: FTG-Net vs Our Model (Prediction Time)

VI. DISCUSSION

FTG-Net [12] is a good model as it is generally fast and achieves good results if the classification window is large enough. However, one of its drawbacks is that it builds a number of flow graphs for each classification window, each containing all the packets exchanged between two endpoints within the classification window. In doing so, it assumes that

the classification occurs in a centralized way. Nonetheless, with high packet rates and the need to perform the classification quickly, distributing the classification becomes inevitable. This means splitting the graph among different workers. This does not guarantee that all packets between the same endpoints are received by the same worker. Therefore, it becomes very challenging to distribute the process due to having a large

number of graphs. That is why designing a new GNN that operates on one graph per classification window is needed in order to allow for future distribution of the classification.

As shown in the previous section, our model achieves satisfactory results for small-sized graphs. However, when the graph size becomes larger, it maintains the high prediction accuracy, but it suffers from large latency. This could be seen as a downside to our model because there are other models like FTG-Net that maintain a low prediction time regardless of the classification window. Upon further inspection of the prediction time, it was observed that the main reason behind the growing latency is the time needed to construct the input graphs. While FTG-Net constructs a large number of smaller graphs, our model constructs one graph for the entire classification window. In fact, if the graph construction time is ignored, our model is faster than FTG-Net by an order of magnitude. This is because it is smaller in size compared to FTG-Net as it is made up of just one GNN, unlike FTG-Net which consists of a hierarchy of GNNs. In order to mitigate this, we define the classification window as a constant number of packets, instead of a fixed time window which may contain a varying number of packets.

Another important observation is related to how the model performs with respect to different attacks. The model is capable of accurately detecting SynFlood and GoldenEye attacks while struggling with SlowLoris and SynScan attacks due to the low presence of these attacks in the dataset. In fact, they make up only 9% of the attacks in the dataset. A possible solution to this is to retrain the model on a dataset containing more examples of these attacks.

VII. CONCLUSION AND FUTURE WORKS

GNNs, designed to work on graph-structured data, have the potential to model complex data patterns, offering a more comprehensive understanding of network traffic. In this work, we proposed a novel GNN-based system for real-time malicious traffic detection, addressing critical challenges such as scalability, inference speed, and practical deployment.

Through experimental analysis, we demonstrated that our model achieves an F1 score of 0.9 or more with a graph configuration that includes a maximum node degree of 50.

Several opportunities exist for enhancing the proposed model. For instance, the hardware acceleration (e.g., GPU or DPU) of the GNN or distributed computing paradigms can be exploited to further reduce the latency. Additionally, expanding the dataset to include a higher number of attacks and traffic patterns could be considered to benchmark the proposed model in a wider range of scenarios. Finally, different aggregation methods for compressing the packets into one node can be tested for possible performance improvements.

edges and a node granularity of 8 packets. Additionally, we determined that accurate detection can be achieved with as few as 50-75 packets for certain attack types. Additionally, we also compared our model to FTG-Net, highlighting the strengths and weaknesses of each one over the other.

ACKNOWLEDGMENT

This work is supported by the Chips Joint Undertaking (JU), European Union (EU) HORIZON-JU-IA, under grant agreement No. 101140087 (SMARTY), including top-up funding by the Italian MUR.

REFERENCES

- [1] M. A. Ferrag, M. Ndhlovu, N. Tihanyi, L. C. Cordeiro, M. Debbah, T. Lestable, and N. S. Thandi, "Revolutionizing cyber threat detection with large language models: A privacy-preserving bert-based lightweight model for iot/iiot devices," 2024.
- [2] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, "Deephunter: A graph neural network based approach for robust cyber threat hunting," in *Security and Privacy in Communication Networks: 17th EAI International Conference, SecureComm 2021, Virtual Event, September 6–9, 2021, Proceedings, Part I* 17, pp. 3–24, Springer, 2021.
- [3] S. Mitra, T. Chakraborty, S. Neupane, A. Piplai, and S. Mittal, "Use of graph neural networks in aiding defensive cyber operations," 2024.
- [4] T. T. Huong, T. P. Bac, D. M. Long, B. D. Thang, N. T. Binh, T. D. Luong, and T. K. Phuc, "Lockedge: Low-complexity cyberattack detection in iot edge computing," *IEEE Access*, vol. 9, pp. 29696–29710, 2021.
- [5] W. L. Hamilton, "Graph representation learning," *Synthesis Lectures on Artificial Intelligence and Machine Learning*, vol. 14, no. 3, pp. 1–159.
- [6] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, and M. Sun, "Graph neural networks: A review of methods and applications," *CoRR*, vol. abs/1812.08434, 2018.
- [7] A. Delpace, S. Hermoso, and K. Anandita, "Cyber attack detection thanks to machine learning algorithms," 2020.
- [8] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, 2014.
- [9] M. Mohammadi, A. Aflaki, A. Kavousifard, and M. Gitizadeh, "Deep learning-based cyber-attack detection model for smart grids," 2023.
- [10] E. Paolini, L. Valcarenghi, L. Maggiani, and N. Andriolli, "Real-time network packet classification exploiting computer vision architectures," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 1155–1166, 2024.
- [11] Y. Li, R. Li, Z. Zhou, J. Guo, W. Yang, M. Du, and Q. Liu, "Graphddos: Effective ddos attack detection using graph neural networks," in *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 1275–1280, 2022.
- [12] L. Barsellotti, L. De Marinis, F. Cugini, and F. Paolucci, "Ftg-net: Hierarchical flow-to-traffic graph neural network for ddos attack detection," in *2023 IEEE 24th International Conference on High Performance Switching and Routing (HPSR)*, pp. 173–178, 2023.
- [13] R. Abu Bakar, L. De Marinis, F. Cugini, and F. Paolucci, "Ftg-net-e: A hierarchical ensemble graph neural network for ddos attack detection," *Computer Networks*, vol. 250, p. 110508, 2024.
- [14] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5g-nidd: A comprehensive network intrusion detection dataset generated over 5g wireless network," 2022.